Data Structures & Algorithms (CS-212)

Week 1: Arrays

Lecture Outline

- Arrays
- Multi-dimension Arrays
- Jagged Arrays
- Self-referential Classes

Arrays

- Elementary data structure that exists as built-in in most programming languages.

```
main( int argc, char** argv )
{
    int x[6];
    int j;
    for(j=0; j < 6; j++)
            x[j] = 2*j;
}
```

Arrays

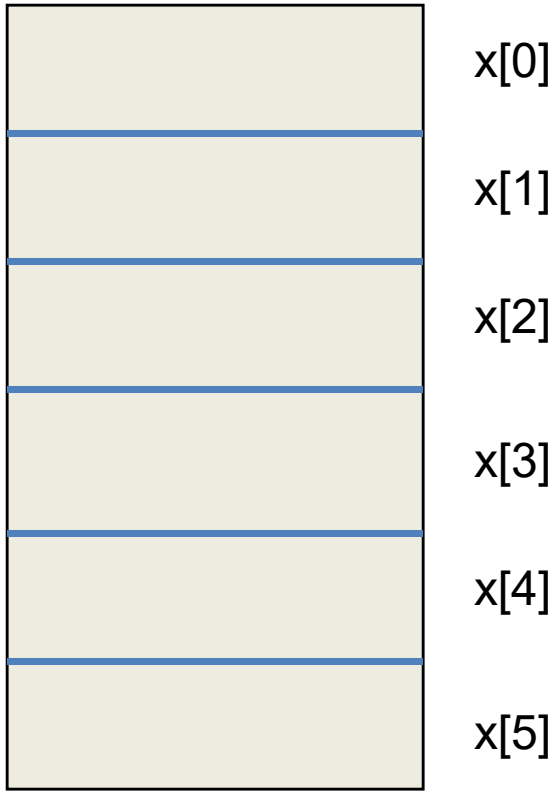
- Array declaration: int x[6];
- An array is collection of cells of the same type.
- The collection has the name ‘x’.
- The cells are numbered with consecutive integers.
- To access a cell, use the array name and an index:

  x[0], x[1], x[2], x[3], x[4], x[5]

Array Layout

Array cells are contiguous in computer memory

The memory can be thought of as an array



What is Array Name?

- ‘x’ is an array name but there is no variable x. ‘x’ is not an *lvalue*.
- For example, if we have the code

```
int a, b;
```

then we can write

```
b = 2;
a = b;
a = 5;
```

But we cannot write

```
2 = a;
```

Array Name

- ‘x’ is not an lvalue

```
int x[6];
int n;

x[0] = 5;
x[1] = 2;

x = 3;          // not allowed
x = a + b;      // not allowed
x = &n;         // not allowed
```

Dynamic Arrays

- You would like to use an array data structure but you do not know the size of the array at compile time.
- You find out when the program executes that you need an integer array of size n=20.
- Allocate an array using the new operator:

```
int* y = new int[20];  // or int* y = new int[n]
y[0] = 10;
y[1] = 15;                    // use is the same
```

Dynamic Arrays

- ‘y’ is a lvalue; it is a pointer that holds the address of 20 consecutive cells in memory.
- It can be assigned a value. The new operator returns as address that is stored in y.
- We can write:

```
y = &x[0];
y = x;          // x can appear on the right
                // y gets the address of the
                // first cell of the x array
```

Dynamic Arrays

- We must free the memory we got using the new operator once we are done with the *y* array.

```
delete[ ] y;
```

- We would not do this to the *x* array because we did not use new to create it.

EXAMPLE: STORE HIGH SCORES FROM A GAME USING ARRAYS

FIRST, THINK OF OBJECT REPRESENTING HIGH SCORE ENTRY

```
class GameEntry {                                   // a game score entry
public:
  GameEntry(const string& n="", int s=0); // constructor
  string getName() const;                           // get player name
  int getScore() const;                             // get score
private:
  string name;                                      // player's name
  int score;                                        // player's score
};
```

Code Fragment 3.1: A C++ class representing a game entry.

```
GameEntry::GameEntry(const string& n, int s) // constructor
  : name(n), score(s) { }
                                                // accessors
string GameEntry::getName() const { return name; }
int GameEntry::getScore() const { return score; }
```

Code Fragment 3.2: GameEntry constructor and accessors.

NEXT, THINK OF A CLASS TO HOLD HIGH SCORES

```
class Scores {                                  // stores game high scores
public:
  Scores(int maxEnt = 10);                      // constructor
  ~Scores();                                    // destructor
  void add(const GameEntry& e);                 // add a game entry
  GameEntry remove(int i)                       // remove the ith entry
      throw(IndexOutOfBounds);
private:
  int maxEntries;                               // maximum number of entries
  int numEntries;                               // actual number of entries
  GameEntry* entries;                           // array of game entries
};
```

Code Fragment 3.3: A C++ class for storing high game scores.

```
Scores::Scores(int maxEnt) {                    // constructor
  maxEntries = maxEnt;                          // save the max size
  entries = new GameEntry[maxEntries];          // allocate array storage
  numEntries = 0;                               // initially no elements
}

Scores::~Scores() {                             // destructor
  delete[] entries;
}
```

Code Fragment 3.4: A C++ class GameEntry representing a game entry.

Only highest scores are retained

Mike	Rob	Paul	Anna	Rose	Jack				
1105	750	720	660	590	510				
0	1	2	3	4	5	6	7	8	9

Figure 3.1: The *entries* array of length eight storing six GameEntry objects in the cells from index 0 to 5. Here *maxEntries* is 10 and *numEntries* is 6.

Insertion

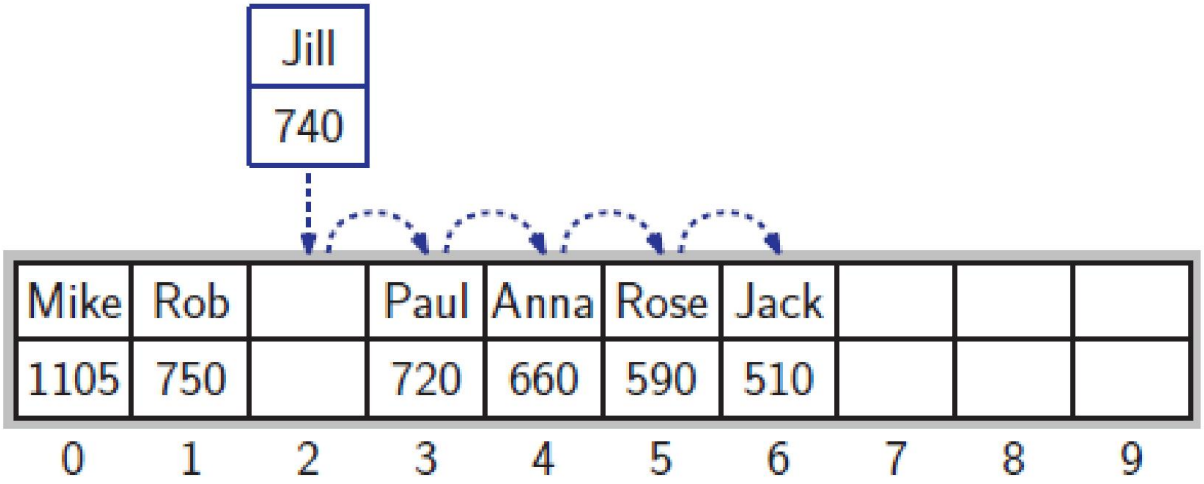


Figure 3.2: Preparing to add a new GameEntry object ("Jill", 740) to the *entries* array. In order to make room for the new entry, we shift all the entries with smaller scores to the right by one position.

Insertion

Mike	Rob	Jill	Paul	Anna	Rose	Jack			
1105	750	740	720	660	590	510			
0	1	2	3	4	5	6	7	8	9

Figure 3.3: After adding the new entry at index 2.

```
void Scores::add(const GameEntry& e) {     // add a game entry
  int newScore = e.getScore();             // score to add
  if (numEntries == maxEntries) {          // the array is full
    if (newScore <= entries[maxEntries-1].getScore())
      return;                              // not high enough - ignore
  }
  else numEntries++;                       // if not full, one more entry

  int i = numEntries-2;                    // start with the next to last
  while ( i >= 0 && newScore > entries[i].getScore() ) {
    entries[i+1] = entries[i];             // shift right if smaller
    i--;
  }
  entries[i+1] = e;                        // put e in the empty spot
}
```

Code Fragment 3.5: C++ code for inserting a GameEntry object.

Insertion

- The number of times we perform the loop in this function depends on the number of entries that we need to shift
- This is pretty fast if the number of entries is small
- But if there are a lot to move, then this method could be fairly slow

Object Removal

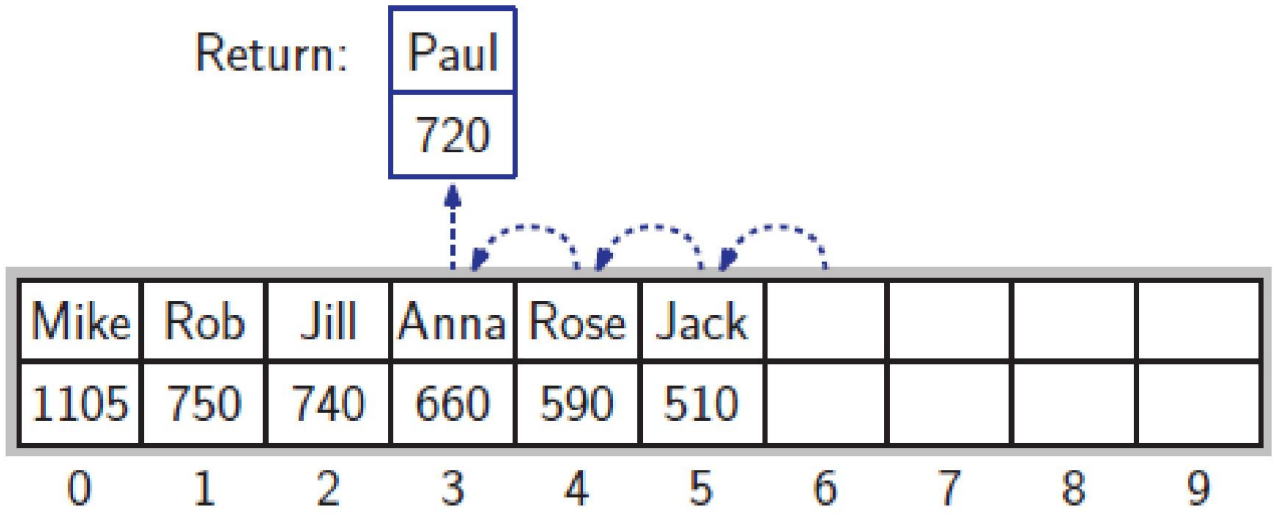


Figure 3.4: Removal of the entry ("Paul", 720) at index 3.

```
GameEntry Scores::remove(int i) throw(IndexOutOfBounds) {
  if ((i < 0) || (i >= numEntries))          // invalid index
    throw IndexOutOfBounds("Invalid index");
  GameEntry e = entries[i];                  // save the removed object
  for (int j = i+1; j < numEntries; j++)
    entries[j-1] = entries[j];               // shift entries left
  numEntries--;                              // one fewer entry
  return e;                                  // return the removed object
}
```

Code Fragment 3.6: C++ code for performing the remove operation.

2-D Arrays

```
int M[8][10];          // matrix with 8 rows and 10 columns
```

	0	1	2	3	4	5	6	7	8	9
0	22	18	709	5	33	10	4	56	82	440
1	45	32	830	120	750	660	13	77	20	105
2	4	880	45	66	61	28	650	7	510	67
3	940	12	36	3	20	100	306	590	0	500
4	50	65	42	49	88	25	70	126	83	288
5	398	233	5	83	59	232	49	8	365	90
6	33	58	632	87	94	5	59	204	120	829
7	62	394	3	4	102	140	183	390	16	26

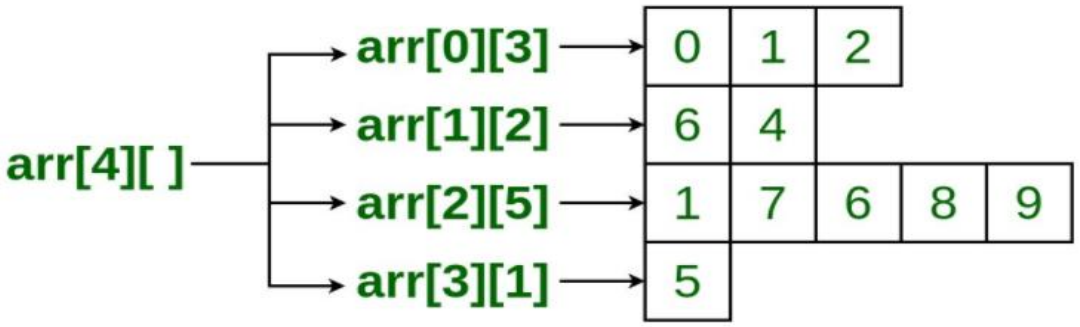
Figure 3.6: A two-dimensional integer array that has 8 rows and 10 columns. The value of $M[3][5]$ is 100 and the value of $M[6][2]$ is 632.

```
cout << M[i][j];       // output element in row i column j
```

Jagged Array

- Jagged array is array of arrays such that member arrays can be of different sizes, i.e., we can create a 2-D array but with a variable number of columns in each row.

```
arr[][] = { {0, 1, 2},
       {6, 4},
       {1, 7, 6, 8, 9},
       {5}
       };
```



int jagged[][] = { {1,2}, {3,4,5} };

```
// Online C++ compiler to run C++ program online
#include <iostream>

int main() {
    // Write C++ code here
    std::cout << "Hello world!";
   int jagged[][] = { {1,2}, {3,4,5} }; // is not permissible


    return 0;
}
```

```
g++ /tmp/UNAsG8lASz.cpp
/tmp/UNAsG8lASz.cpp: In function 'int main()':
/tmp/UNAsG8lASz.cpp:7:8: error: declaration of 'jagged' as
    multidimensional array must have bounds for all dimensions except
    the first
    7 |    int jagged[][] = { {1,2}, {3,4,5} }; // is not permissible
      |        ^~~~~~
```

Jagged Array in C++

- Static Jagged Array
- Dynamic Jagged Array

Static Jagged Array

- Using array and a pointer
- Steps:
  1. Declare 1-D arrays with the number of rows you will need,
  2. The size of each array (array for the elements in the row) will be the number of columns (or elements) in the row,
  3. Declare a 1-D array of pointers that will hold the addresses of the rows,
  4. The size of the 1-D array is the number of rows you want in the jagged array.

Implementation in C++

```
#include <iostream>
#include <string>
int main()
{
    int row0[4] = { 1, 2, 3, 4 };
    int row1[2] = { 5, 6 };
    int* jagged[2] = { row0, row1 };
    int Size[2] = { 4, 2 }
int k = 0;
   for (int i = 0; i < 2; i++) {  // rows
          int* ptr = jagged[i];

        for (int j = 0; j < Size[k]; j++) { // column
            std::cout<<*ptr<<" ";
            ptr++;
        }

        std::cout<<"\n";
        k++;
        jagged[i]++;
    }
    return 0;
}
```

Output

1 2 3 4

5 6

Dynamic Jagged Array

- Using an array of pointer
- Steps:
  1. Declare an array of pointers (jagged array),
  2. The size of this array will be the number of rows required in the Jagged array
  3. For each pointer in the array allocate memory for the number of elements you want in this row.

```
#include <iostream>
#include <string>
int main()
{
    // 2 Rows
    int* jagged[2];

    // Allocate memory for elements in row 0
    jagged[0] = new int[1];

    // Allocate memory for elements in row 1
    jagged[1] = new int[5];

    // Array to hold the size of each row
    int Size[2] = { 1, 5 }, k = 0, number = 100;

    // User enters the numbers
    for (int i = 0; i < 2; i++) {

        int* p = jagged[i];

        for (int j = 0; j < Size[k]; j++) {
            *p = number++;

            // move the pointer
            p++;
        }
        k++;
    }
    k = 0;

    // Display elements in Jagged array
    for (int i = 0; i < 2; i++) {

        int* p = jagged[i];
        for (int j = 0; j < Size[k]; j++) {

            std::cout<< *p<<" ";
            // move the pointer to the next
element
            p++;
        }
        std::cout<<"\n";
        k++;
        // move the pointer to the next
row
        jagged[i]++;
    }

    return 0;
}
```

Output

100

101 102 103 104 105

Self Referential Classes

- A self-referential class contains a pointer member that points to a class object of the same class type

```
// A class can have a pointer to self type
#include<iostream>

using namespace std;

class Test {
  Test * self; //works fine

  /* other stuff in class*/

};

int main()
{
  Test t;
  getchar();
  return 0;
}
```

Conclusion

- These functions for adding and removing objects in an array of high scores are simple
- Nevertheless, they form the basis of techniques that are used repeatedly to build more sophisticated data structures

Conclusion

- These other structures may be more general than our simple array-based solution, and they may support many more operations
- But studying the concrete array data structure, as we are doing now, is a great starting point for understanding these more sophisticated structures, since every data structure has to be implemented using concrete means

Lecture content adapted from Michael T. Goodrich textbook, chapters 3.